

型検査パス — 実行する前に間違いを見つける

今日のゴール

コード生成の前に AST を検査して、未定義の変数・未定義の関数・引数の個数違い・代入できない左辺を見つけるパスを書く。誤りは 1 件で止めず、まとめて報告する。

1 この回の位置づけ

- 品質発展シリーズの第 1 回（選択制）。前提はコマ 12 まで
- `mycc.py` は書き換えない。ラッパー `checkcc.py`（完成品）が Parser の出口に検査を差し込む
- 編集するのは `typecheck.py` の 3 つのメソッドだけ

2 いまのコンパイラは何を見逃しているか

次のプログラムを、いまのコンパイラに食わせてみる。

```
int f(int a, int b) { return a + b; }
int main() {
    int x;
    x = y + 1;          /* y なんて変数はない */
    f(1);               /* f は引数 2 個のはず */
    g(2);               /* g という関数はない */
    1 + 2 = 3;          /* 1 + 2 に代入はできない */
    return x;
}
```

結果は、たとえばこうなる。

```
RuntimeError: [line 4] 未定義の変数 (type_of): 'y'
Traceback (most recent call last):
  File ".../mycc.py", line 855, in gen_program
  ...
```

問題が 2 つある。

1. 1 件目で止まる。y を直しても、次は f(1) で止まる。1 つずつしか進めない
2. エラーがコード生成の都合で出ている。「型を調べようとしたら見つからなかった」という内部の事情であり、g(2) や 1 + 2 = 3 に至っては、運が良ければ動いてしまう（間違ったコードが生成される）

原因は、検査とコード生成が混ざっていることである。コード生成は「正しいプログラムが来る」前提で書かれているので、誤りへの対応が場当たりになる。

3 独立したパスにする

そこで、コード生成の前に、検査だけを行うパスを置く。

ソース → 字句解析 → 構文解析 → [型検査] → コード生成 → アセンブリ
↓
誤りがあれば全部報告して終了

この配置には、はっきりした利点がある。

利点	理由
誤りをまとめて報告できる	木を最後まで歩いてから結果を返せる
メッセージが分かりやすい	「未定義の変数 y」 — 内部事情が混ざらない
コード生成が単純なまま	「正しい木しか来ない」前提を守れる

実際のコンパイラでは、この段階を意味解析（semantic analysis）と呼ぶ。型検査はその中心的な仕事である。

4 何を検査するか

この回で扱うのは 4 種類である。どれも「AST を見れば分かる」ものに絞ってある。

検査	例	メッセージ
未定義の変数	<code>x = y + 1;</code>	未定義の変数: <code>y</code>
未定義の関数	<code>g(2);</code>	未定義の関数: <code>g</code>
引数の個数違い	<code>f(1)</code> (<code>f</code> は 2 引数)	関数 <code>f</code> の引数の個数が合いません (宣言 2 個、呼び出し 1 個)
代入先が lvalue でない	<code>1 + 2 = 3;</code>	代入先にできない式です (Add)

最後の 1 つは、コマ 9 で学んだ **lvalue / rvalue** の区別そのものである。`codegen_lval()` が扱えるノード (`Var / Deref / Index / Member`) だけが代入先になれる、という規則を検査に写す。EBNF の注釈「lvalue 制約は意味解析フェーズで検査」(F2 で触れた箇所) が、ここで実装される。

5 スコープの持ち方

変数を探すときは、コマ 14 と同じ順序で調べる。

locals (いまの関数) → globals (プログラム全体) → 見つからなければエラー

`collect()` (完成済み) がトップレベルを一巡して `globals` と `funcs` の一覧を作り、関数ごとに `locals` を作り直す。関数の一覧を先に作るので、後ろで定義された関数を前から呼んでも誤検出しない。

重要

引数の個数进行检查してよいのは、定義がこのプログラム内にある関数だけである。

`int printf(char *fmt, ...);` のような宣言だけの関数は、可変長引数かもしれない。しかもスキヤフォールドの Parser は `...` を読み飛ばしてしまい、**AST** に可変長かどうかの情報が残らない (F4 で見た「何を AST に残すかも設計」の実例)。そこで `FuncDef` (定義) がある関数だけ個数进行检查する。

「確実に言えることだけ言う」— 検査を作るときの基本姿勢である。誤検出 (正しいプログラムをエラーにする) は、見逃しよりたちが悪い。

6 実装

Step	実装対象	内容
1	<code>check_var</code>	未定義の変数
2	<code>check_call</code>	未定義の関数・引数の個数
3	<code>check_assign</code>	代入先が lvalue か

収集（collect）と木の走査（walk）は完成済みで、書くのは「何を誤りとみなすか」だけである。

```
# 検査だけする
python3 checkcc.py --check-only tests/undefined_var.c

# 検査してから、問題なければコンパイルする
python3 checkcc.py ../../../../final/tests/f09_recur.c
```

7 テスト

```
python3 check.py
```

2 種類の確認をする。

1. エラーコーパス: `tests/*.c` を検査し、報告内容が `tests/*.expected` と一致するか（行番号とメッセージまで一致させる）
2. 正常系: 講義のテスト入力（約 96 ファイル）を検査し、**1 件も誤検出しないこと**

2 番目が重要である。検査は厳しすぎても使えない。「正しいプログラムを 1 つも拒まない」ことを、既存の資産で保証する。

8 発展課題

1. **型の不一致を見つける:** `int` に `int *` を代入している箇所を報告する。ただし `malloc` の戻り値のように、意図的に許している変換もある (`language_spec.md` の「`void *` は暗黙変換可」)。どこまで厳しくするか設計する
2. **未定義メンバの参照:** `p->nonexistent` を検出する (コマ 12 の `_struct_defs` を検査側でも作る必要がある)
3. **未使用変数の警告:** 宣言されたが一度も読まれない変数を報告する。エラーではなく警告として、別の一覧にする
4. **`return` の検査:** 戻り値型が `int` の関数で、値なしの `return;` を報告する。逆に `void` 関数で値を返している場合も
5. **エラーの上限:** 誤りが 100 件を超えたら「多すぎます」と打ち切る (壊れた入力で数千件出ても読めない)

補足

コラム: 型検査はどこまでやるべきか。C の型検査は緩い (`int` とポインタの混同を警告止まりにする処理系も多い)。一方 Rust や Haskell は、型検査を通ったプログラムはある種の誤りを決して起こさないことを保証する。検査を厳しくするほど早く誤りを見つけられるが、正しいプログラムまで拒む危険と、書き手の負担が増える。この綱引きが言語設計の中心的な論点になっている。

いま書いたパスは、C コンパイラが出す診断の中で最も基本的なもの (`undeclared identifier`、`too few arguments`、`lvalue required as left operand of assignment`) に対応している。